Language Resources and Evaluation https://doi.org/10.7707/s10349-024-09793-1

ORIGIINAL PAPER

Collaborative Web-based Framework for NLP Pipeline Assessment

Nerissa Lombardi[1] · Jack Murphy[1] · Amira S. Ezzat[1]

[1] Center for Computational Language Science, University of Leuven, Jamova 39, 1000 Ljubljana, Slovenia

Accepted: 20 December 2024
© The Author(s) 2024

Abstract

The systematic assessment of natural language processing (NLP) technologies demands not only accessible workflow tools but also robust platforms for experimental reproducibility and component reuse. We introduce FlowBench, a user-oriented, web-based framework that enables the rapid construction, execution, and benchmarking of open-source NLP pipelines. Designed with interoperability at its core, FlowBench streamlines integration and comparative analysis of popular tools by supporting drag-and-drop assembly, automatic evaluation modules, and transparent workflow sharing across research teams. To validate its utility, FlowBench was used to integrate and benchmark six leading part-of-speech taggers across five annotated language corpora, with results demonstrating reproducible ranking and compatibility for future algorithmic extensions. Features such as pre-built evaluation flows and experiment provenance tracking foster open science and empower developers to reuse and extend pipeline modules efficiently. The influence of factors like corpus size and pre-training is further explored, showcasing the platform's flexibility in isolating experimental variables while promoting best practices in NLP evaluation.

Keywords Open source · Online platform · Replicability and reusability · NLP evaluation · POS tagging

1 Introduction

Evaluation of tools has always been an important issue in the natural language processing (NLP) community, which is demonstrated by the organization and increasing popularity of shared NLP tasks (e.g., CoNLL, MUC, CLEF, PAN). Nevertheless, an open science approach to a systematic evaluation of different NLP tools in an open source web-based environment has not yet been made available for an online experimentation. While there has been some attempts to develop platforms for online evaluation of specific tools, e.g., the General Entity Annotator Benchmarking (GERBIL) framework for evaluation of named entity recognition (NER) tools (Usbeck et al., 2015), we are not aware of any online platform that was used as a general evaluation framework for newly developed NLP tools of different kinds.

The research presented in this paper fills this gap by exploring one possible solution to this problem, that is, the development of evaluation workflows in the TextFlows[1] platform Perovšek et al. (2016). TextFlows is a web-based platform that focuses on facilitating the construction and execution of NLP and text mining workflows. It requires no local installation and is simple to use. The distinguishing feature of TextFlows is the ease of sharing and publicizing NLP workflows, together with a roster of reusable workflow components and entire workflows. As completed workflows, data, and results can be made public by the author, the platform can serve as an easy-to-access integration platform for text mining and NLP. Each public workflow is assigned a unique URL that can be accessed by anyone to either replicate the experiment or use the workflow as a template to design similar new workflows.

TextFlows platform is open source (source code is freely available on GitHub and the instructions for adding widgets are included), which means that new components (e.g., NLP tools) that a user wishes to evaluate can be easily integrated in the platform, making the platform flexible enough to be used as an evaluation framework for newly developed tools.

The main contributions of this research are the following.

- Exploration of the TextFlows platform as an online NLP evaluation framework. More specifically, we describe the approach that facilitates the integration of new NLP tools and building of publicly available and reproducible evaluation workflows.
- Development of several evaluation workflows, offering thorough evaluation and comparison of seven part of speech taggers in two different evaluation scenarios – intrinsic and extrinsic – with the aim of showcasing the appropriateness of the platform for a given task. We also demonstrate that the NLP algorithm integration in the online platform TextFlows enable transparent and replicable experimentation using the precomposed workflows.
- Integration of several new tools needed for our experiments into the TextFlows platform, including three new POS taggers and components for corpus loading

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

and format conversion. These components are now publicly available in the platform for others to reuse.

In relation to the prior work in which we presented the first NLP experiments concerned with extrinsic evaluation of POS taggers in the TextFlows platform (Perovšek et al., 2015), this paper is more detailed and presents many new experiments. Besides integrating several new part-of-speech (POS) taggers in the platform, this study showcases, that besides the extrinsic evaluation conducted in our previous work, TextFlows is also appropriate for the intrinsic POS tagger evaluation. More specifically, using taggers as components in the lemmatizer/stemmer evaluation experiments, we address the intrinsic evaluation of seven POS taggers on six gold standard data sets in a completely reproducible setting, with all implemented workflows published online.

The paper is structured as follows. Section 2 presents the related work. Section 3 presents the TextFlows platform, followed by several sections that demonstrate the utility of the TextFlows platform in a selected POS tagger evaluation use case. Section 4 presents the motivation for selecting this use case. Section 5 presents the intrinsic evaluation of numerous publicly available POS taggers on a number of freely available POS tagged gold standard corpora, followed by the extrinsic evaluation of POS taggers, both performed using publicly accessible workflows developed in the TextFlows platform. Section 6 presents the conclusions, including the platform limitations and the directions for its further development.

2 Related work

Following the recent focus on reusability and replicability in natural language processing (NLP) experiments, the tools have started being integrated into online platforms and made available as web services. For example, the CLARIN European Research Infrastructure (Eskevich et al., 2020; Jong et al., 2022) is being developed, aiming at providing unified single-sign-on access to language resources and advanced NLP tools. Among the CLARIN developments, WebLicht (Hinrichs, Zastrow, and Hinrichs, 2010; Zinn & Campbell, 2023) is oriented towards corpus processing and offers a large number of NLP components, although it is not open source. WebLicht does implement a user interface for workflow construction, but these workflows (and experiments conducted with these workflows) cannot be published and shared online for simple reproducibility. Besides English, WebLicht also has several components for German NLP and a limited number of components for other languages.

The Language Grid (Ishida, 2006) multilingual internet service platform is open source and supports a web-oriented version of the pipeline architecture typically employed by NLP tools, but the platform is quite complex to install and use, partially due to its age, as it uses somewhat outdated technologies. This also means that the platform does not offer support for never (mostly neural) models. However, the platform supports several languages and allows workflow construction and sharing.

GATE (Cunningham, 2002) is another NLP-focused platform that offers a large suite of NLP tools for several languages. The platform offers a user interface in which text can be processed and analyzed with a set of interactive graphical tools. The platform also enables limited sharing of built pipelines by sending them to GATE Cloud or GATE Teamware, a web-based collaborative annotation platform (Bontcheva et al., 2013). However, these workflows cannot be publicly published, which also limits the reproducibility of the experiments conducted with these workflows. The platform also enables the integration of existing tools, such as, for example, LingPipe (Carpenter, 2007).

In contrast to GATE, ScispaCy (Neumann, King, Beltagy, and Ammar, 2019) is a platform that requires coding skills for the creation of scientific pipelines and only supports English. The platform offers English named entity recognition (NER) models, part-of-speech (POS) tagging models, dependency parsing, and rule-based tokenization. It also supports external libraries, such as Tensorflow, for deep learning. The usage of the platform nevertheless requires a high level of technical skills, since it does not have a user-friendly graphical user interface (GUI). It also does not allow workflow sharing.

OpenMinTed (Labropoulou et al., 2018) is an NLP platform built to support the analysis of scientific articles. The platform is made up of three modular blocks, "Content resources" containing open access articles used as input for text and data mining software, "Text and data mining software" for basic NLP tasks (e.g., tokenization, sentence splitting and NER), and "Ancillary knowledge resources" containing machine learning models, terminologies, and tools for the management of ontologies. The built workflows can be executed within the Galaxy platform (Afgan et al., 2018), a workflow management system for biomedical analyzes with a graphical user interface.

Another platform dedicated to the construction of NLP workflows is LAPPS Grid (Ide et al., 2015). It consists of a web interface, which contains a wide range of NLP tools coming from multiple providers. The platform does not have a GUI and also lacks several general NLP components, such as POS taggers.

Some platforms have also been developed for the specific purpose of deployment of clinical NLP pipelines. Given that in medicine, the reproducibility of experiments is of utmost importance, is becoming challenging due to the increased complexity of data streams and computational processes in modern clinical health information systems (Digan et al., 2021). The cTakes (Savova et al., 2010) platform, for example, is deployed in several American hospitals and universities. It is open source, relies on the UIMA[2] model, and is released by default with a clinical NLP pipeline. It supports common NLP operations, such as tokenization and NER. The pipelines can be manipulated and executed through command-line and graphical interfaces. Another clinical NLP solution is CLAMP (Soysal et al., 2018), which is also based on UIMA models. It has a graphical user interface that allows for the creation of NLP pipelines by a simple drag-and-drop of integrated NLP components. It can also be used from the command line. Besides just specialized clinical NLP components and tools for

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

corpus management and annotation, this platform also contains several machine learning components (e.g., support vector machine, rule-based methods, etc.) that can be retrained on a new corpus. It does not support neural models.

The largest and most important ongoing initiative is European Language Grid (ELG)[3] (Rehm, Berger, et al., 2020; Rehm et al., 2021), Rehm et al. (2021). The platform contains over 1,300 commercial and non-commercial tools and services, covering all European languages, and thousands of language resources. The system aims to become the primary platform for language technologies in Europe by enabling the show casing, sharing, and distribution of tools, services and resources from research and industry. The platform also has ambitions of becoming a marketplace for all European language technology products, services and organizations, and to support the digital language equality in Europe, by curbing the current predominance of technologies that support English. While the platform's biggest advantage is its large repository of NLP components (among them also state-of-the-art neural models), it has limitations when it comes to workflow sharing.

Text processing components are also supported in some more general machine learning platforms, such as Orange[4](Demšar et al., 2013), RapidMiner (Hofmann & Klinkenberg, 2016) and KNIME (Delen, Helfrich, and Silipo, 2021). Especially the latter platform contains components for all basic NLP operations (e.g., tokenization, POS tagging, NER), allows workflow construction and sharing, and also contains some state-of-the-art neural models, therefore, making it quite useful for NLP. For a detailed comparison of the above-mentioned platforms across several dimensions, see Sect. 3.7.

The scientific community also recently showed interest in the development of NLP platforms. In 2020, an ACL workshop on NLP platforms was organized (Rehm, Bontcheva, et al., 2020), where a wide range of platforms was presented Branco et al. (2020), Coleman et al. (2020), Cristea et al. (2020), Etchegoyhen et al. (2020), Geißler (2020), Moreno-Schneider et al. (2020) and Skadiņš and Salimbajevs (2020). However, the majority of the proposed platforms do not adopt the visual programming paradigm, which is one of the features of TextFlows, an NLP platform we use for our experiments.

Finally, we are aware of one attempt to build an online evaluation framework. General Entity Annotator Benchmarking (GERBIL) (Usbeck et al., 2015) is an evaluation framework for semantic entity annotation. It provides an uniform evaluation of named entity annotation tools on multiple datasets to developers, end users and researches, allowing them to compare newly developed tools to state of the art. The evaluation experiments are available on permanent URIs, ensuring the reproducibility and archiving of evaluation results. The platform also provides some diagnostic tools offering insights into inner-working of the tools and a simple user interface for the provision of evaluation results.

3 The TextFlows platform

The TextFlows platform Perovšek et al. (2016) is a cloud-based open-source web application for the composition, execution and sharing of interactive text mining workflows. The NLP tools the platform contains are largely monolingual and only support English text processing. TextFlows is a fork of ClowdFlows Kranjc et al. (2012); Žnidaršic et al. (2024, submitted), a more generic cloud-based data mining platform. It has a web-based user interface for building workflows and requires no local installation since it runs in all major web browsers. It contains many workflow components and is simple to use and debug because of the visual programming approach to workflow composition.

3.1 TextFlows architecture

In software engineering, the terms *front-end* and *back-end* are used to distinguish the separation between a presentation layer (the client side) and a data access layer (the server side), respectively. Figure 1 shows the TextFlows architecture, which is logically separated into the front-end (in pink) and the back-end (in blue). The back-end comprises a relational database for storing workflows, workers for task execution, and a broker for delegating tasks from different queues to workers which can reside on different clusters of machines. The front-end is designed for user interaction with workflows through the graphical user interface in a web browser.

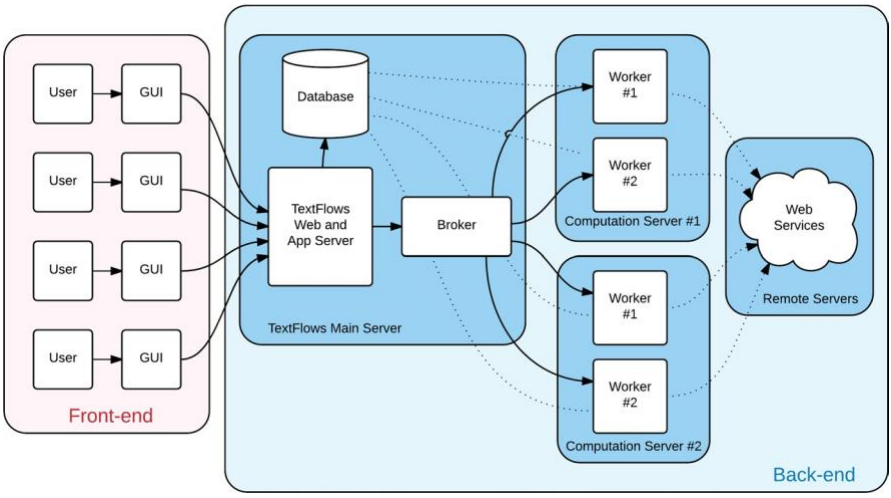

Fig. 1 An overview of the TextFlows architecture, separated into the front-end (in pink) and the back-end (in blue)

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

3.1.1 The platform back-end

The back-end of the TextFlows platform uses Django,[5] a Python-based open-source high-level web framework. Django follows the model-view-controller architectural design, which encourages rapid developement, and includes an object-relational mapper and a template engine for efficient code generation. The object-relational mapper provides an interface which links objects to a database. PostgreSQL is used in the public installation of TextFlows.

Without additional extensions, Django is synchronous, sometimes also described as blocking, meaning that an HTTP request will not be returned until all processing is complete. However, in TextFlows we need tasks to run in the background without blocking, and different system environment requirements of implemented libraries dictate that the TextFlows platform must be distributed across multiple machines. To this end, TextFlows task queues are used as a mechanism to distribute work across threads and machines. This is performed via Celery,[6] which is a task queue system based on distributed message passing. Celery communicates through messages and uses a message broker to mediate between clients and workers. To initiate a task, a client adds a message to the queue, which the broker then sends to a worker. The system used as a broker in TextFlows is RabbitMQ,[6] a complete and highly reliable enterprise messaging system based on the Advanced Message Queuing Protocol (AMQP). It offers not only exceptionally high reliability, but also high availability and scalability, which is vital for the TextFlows platform.

TextFlows uses the PySimpleSoap library[7] for integrations of web services as workflow components. PySimpleSoap is a lightweight Python library, which provides an interface for client and server web service communication. Using PySimpleSoap we can not only import WSDL web services as workflow components, but also expose entire workflows as WSDL web services.

3.1.2 The platform front-end

The client side of the TextFlows platform consists of operations that involve user interaction primarily through the graphical user interface (GUI) in a modern web browser. The graphical user interface is implemented in HTML and JavaScript, with extensive use of the jQuery library.[8] On top of jQuery we use the interaction library jQuery-UI,[9] which is a collection of GUI modules, animated visual effects, and themes. This library supports the option to make elements in the graphical user interface draggable, droppable, and selectable, which are the features supported by the TextFlows workflow canvas.

3.2 Workflows and widgets

Data processing in TextFlows is managed by connecting processing components (named widgets) into an executable pipeline on a design canvas. The graphical user interface for constructing the executable pipelines (named workflows) follows a visual programming paradigm that simplifies the representation of complex procedures into spatial arrangements of their building blocks (Fig. 2), so building complex scientific workflows made out of many (pre-integrated) NLP tools becomes a matter of dragging and dropping workflow components on the canvas and connecting them.

Each basic processing unit (graphically represented as a widget) has a predefined set of inputs and parameters, performs a task, and stores the results as outputs. A partial menu of different text processing widgets is shown in Fig. 3.

Different processing components are linked via connections through which data is transferred from a widget's output to another's input. Some widgets take as input user parameters entered directly into the widget's text fields.

The GUI implements an easy-to-use formalism of arranging widgets on a canvas to form a graphical representation of complex procedures. Construction of new workflows requires no expertise apart from knowing (usually from widget documentation) the inputs and outputs of widgets to ensure their compatibility. Once constructed, the workflows can be shared (publicly or privately), reused, and extended.

3.3 Workflow sharing

Workflows in TextFlows are processed and stored on remote servers from where they can be accessed from anywhere, requiring only an internet connection. By default, each workflow can only be accessed by its author, although one may also choose to make it publicly available. The TextFlows platform generates a

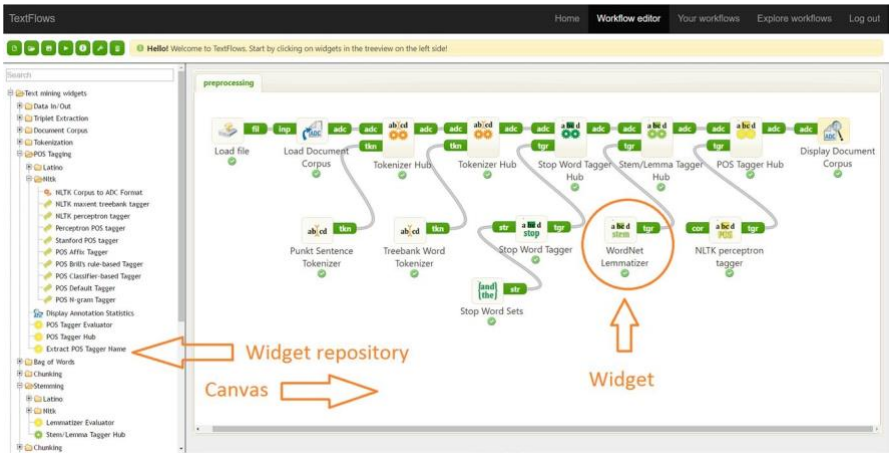

Fig. 2 An illustrative preprocessing workflow incorporated in the TextFlows canvas and a menu of widgets, which can be used as workflow components

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

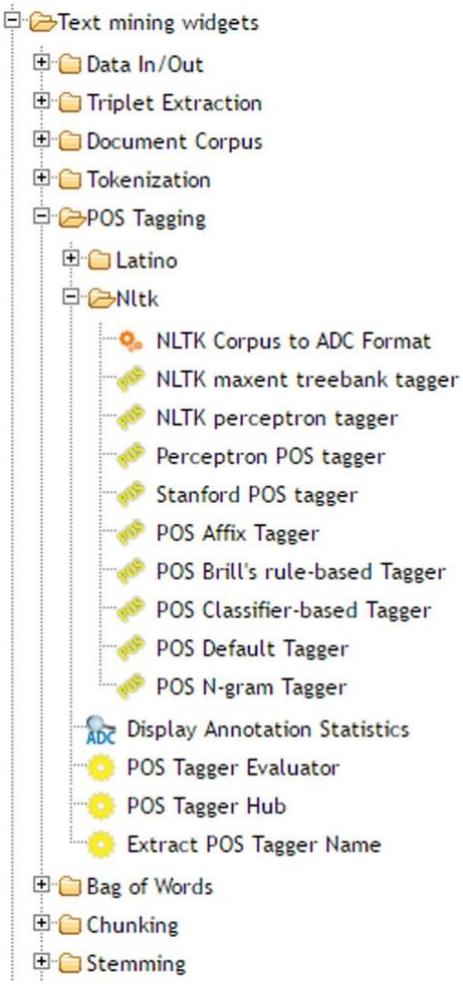

Fig. 3 Selected NLP processing components available in the TextFlows widgets menu, most relevant to POS tagger evaluation

specific URL for each workflow that has been saved as public. The users can then simply share their workflows by publishing the URL.

Whenever a public workflow is accessed by another user, a copy of the workflow is created on the fly and added to his private workflow repository. The workflow is copied together with widgets' parameter settings, as well as all the data, in order to ensure the experiments can be repeated. This way the user is able to tailor the workflow to his needs without modifying the original workflow.

The platform, ideal for visual representation of complex procedures, allows the replication of experiments. When the users create their workflows, they can publish them together with the data and the results, making them available to a wider audience. All the experiments presented in this paper are therefore available online.

3.4 Integration of new components

TextFlows is released under the MIT license, which allows it to be modified and its modifications made public. The source code is available on GitHub.[10] Using Git, it is possible to download the source code to a development machine where a local installation of TextFlows can be set up.

In the local adaptation of TextFlows, widgets are contained in packages, which are collections of widgets that perform similar tasks. In the technical sense, a TextFlows package is a collection of two types of files — widget source codes and package metadata. The source code determines what kind of task the widget performs. The metadata for each widget is automatically generated using a form in the TextFlows administration panel, which is accessible from the local development version. Each TextFlows package includes a file entitled "library.py". This file contains the source code of the widgets. Each widget is represented by a static method in this file. Each method receives a dictionary (a native Python data type) of inputs and should return a dictionary of outputs. Note that the keys in the input and output dictionaries can be arbitrary, i.e. meaning that the semantic interoperability of different widgets is not enforced and there is no assurance that two distinct widgets can be directly connected in the workflow. The semantic interoperability is however encouraged through the use of the AnnotatedDocumentCorpus (ADC) format (see Sect. 3.5 for details), a collection of documents and its meta data, used as a default input/output format.

When integrating new components (e.g., in our case POS taggers and lemmatizers), these components need to be wrapped in a method inside a "library.py" file in a specific appropriate package and presented as widgets. The inputs, outputs, widget name, description, and other meta data are entered in the TextFlows administration panel. When a widget is created in the administration panel it can immediately be used in the local TextFlows installation. To transfer a widget from one TextFlows installation to another (e.g., from a development version to a live production version) the meta data files need to be created. They are created using the TextFlows package manager, which provides two command line tools: one for importing packages and another for exporting them. The export package command on the development server examines all widgets in the database (these were either imported or entered using the administration panel) and compares them to widgets in the data files. Missing widgets are added to the files (or updated if changed). On the target TextFlows instalation the import command must be executed. Conversely, this command compares data files with the database and adds (or removes if desired) widgets to the database.

3.5 Representation framework

The key concepts in text mining are a corpus or a document collection, a single document, and document features (Feldman & Sanger, 2007). Below we describe the model of corpora, documents, and annotations on documents in TextFlows. When

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

designing TextFlows, the emphasis was on providing common representations that are passed among the majority of widgets:

Annotated corpus. A document collection is any grouping of text documents to be used for text analytics purposes. In TextFlows, the Python class that represents a corpus of documents is called *AnnotatedDocumentCorpus (ADC)*. An ADC instance contains its collection of documents and its metadata such as the authors, creation date, facts and notes about the dataset, etc. Features are stored in a simple keyvalue Python dictionary, where keys are strings and the values can store any Python object.

Annotated document. A single textual data unit within a collection—a document—is represented by the class *AnnotatedDocument*. An *AnnotatedDocument* object may vary in size from a single sentence to a whole book. As with ADC, *AnnotatedDocument* instances also contain meta-data, such as the author, date of publication, document length, assigned keywords, etc.

Annotation. Instances of the *Annotation* class are used to mark parts of the document, e.g., words, terms, or sentences. Each *Annotation* instance has two pointers, one to the start and one to the end of the annotated stretch in the document text. These instances also have a type attribute used for grouping annotations of a similar nature and contain keyvalue dictionaries of features, used by taggers to annotate parts of a document with specific tags, e.g., annotations of type "token" that have a feature named "StopWord" with value "true", represent stop words in the document.

3.6 Repository of NLP widgets

Widgets are stored in the Textflows widget repository, where they are grouped and described according to their function. The main groups of widgets are described below.

Corpus and vocabulary acquisition. Document acquisition is usually the first step of every text mining methodology. TextFlows employs widgets that enable loading document corpora, labeling of documents with domain labels and converting them into the ADC structure. Document corpora can be loaded from files, where the dataset can be either a single textual file, with each line representing a separate

document, or a zip of files in which a document is represented as a file. Also supported is the upload of Word (.doc or.docx) and PDF files. Together with the document text, the files may optionally contain document meta-data.

Corpus manipulation and visualization. TextFlows implements several widg- ets for the manipulation of ADC data objects. These widgets allow the user to add new features, extract existing features from the document corpus, split document corpora (by either specifying conditions or by indices), merge different corpora, etc. A special widget in the platform is the interactive *Document Corpus Viewer* widget, which visualizes the ADC data objects. The *Document Corpus Viewer* widget allows the user to check the results of individual widgets by visualizing the ADC data object from their outputs.

Text preprocessing widgets. Preprocessing is a very important part of any form of knowledge extraction from text documents. Its main task is the transformation of unstructured data from text documents into a predefined well-structured data representation by extracting a high-quality feature vector for every document in a given document corpus. The implementation employs the LATINO,[11] scikit-learn (Pedregosa et al., 2011a) and NLTK (Natural Language Toolkit) (Bird, 2006) software libraries for its text preprocessing needs. Among others, these libraries contain the majority of elementary text preprocessing procedures as well as advanced procedures that support the conversion of a document corpus into a table of instances, thus

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

converting every document into a table row representation. The TextFlows preprocessing techniques are based on standard text mining concepts (Feldman & Sanger, 2007) and are implemented as separate categories. Every category possesses a unique hub widget, which has the task of applying a preprocessing technique from its category to the ADC data object. Every such widget is library-independent, meaning that it can execute objects from either LATINO, NTLK or scikit-learn libraries. A standard collection of preprocessing techniques integrated in TextFlows includes: tokenization, stopword removal, POS tagging, as well as stemming and lemmatization.

Document classification. Document classification (also referred to text categorization) concerns the automated assignment of text documents to predefined categories based on their content. In the TextFlows platform, there are several implemented widgets representing different classifiers from the LATINO (Nearest Centroid Classifier, Naive Bayes Classifier, Maximum Entropy Classifier, k-Nearest Neighbors Classifier, SVM Classifier), NTLK (Naive Bayes Classifier) and scikit-learn (Gaussian Naive Bayes Classifier, k-Nearest Neighbors Classifier, SVM Classifier, Decision Tree Classifier, Logistic Regression Classifier) libraries. Every widget contains input parameters that are used to construct the classifier. For training and applying classifiers, there are two widgets in the

Table 1 Comparison of TextFlows to other similar NLP platforms according to seven dimensions

Platform	Supported languages	Licensing	GUI	Workflow construction	Workflow sharing	Deep learning support	POS taggers available
TextFlows	English	MIT	Yes	Yes	Yes	No	Yes

WebLicht	Multiple languages	/	Yes	Yes	No	No	Yes
Language Grid	Multiple languages	GPL2	Yes	Yes	Yes	No	Yes
GATE	Multiple languages	LGPL¯3.0	Yes	Yes	No	No	Yes
ScispaCy	English	Apache	No	Yes	No	Yes	Yes
OpenMinTed	English	Apache	No	Yes	Yes	No	Yes
LAPPS Grid	English	Apache	No	Yes	Yes	No	No
cTakes	English	Apache	Yes	Yes	Yes	No	Yes
CLAMP	English	/	Yes	Yes	Yes	No	Yes
ELG	Multiple languages	/	Yes	Yes	No	Yes	Yes
KNIME	Multiple languages	GNU	Yes	Yes	Yes	Yes	Yes

TextFlows platform: *Train Classifier Hub* and *Apply Classifier Hub*. The *Train Classifier Hub* receives on its inputs a sparse feature dataset object and an untrained classifier. Its function is to fit the classifier model according to the given training data. The final outcome of this widget is a trained classifier object. The *Apply Classifier Hub* receives such an object and returns predicted class probabilities for every new document from the input test dataset, as well as the test dataset with a new predicted labels column.

Evaluation and visualization. The TextFlows platform enables users to create interactive charts for easy and intuitive evaluation of performance results. It includes standard performance visualizations used in machine learning, data mining, information retrieval, etc. The platform contains several visual performance evaluation widgets, which result in interactive performance charts that can be saved and exported to several formats: scatter charts (ROC space charts and PR space charts), curve charts (PR curves, Cost curves, Lift curves, ROC curves,
Kendall curves, and Rate-driven curves), and general column charts for visualizing multiple performance measures for a set of algorithms.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

3.7 Comparison to other platforms

As explained in Sect. 1, we opted to use TextFlows for the evaluation experiments because of the ease of sharing public NLP workflows and due to the fact that the platform is open source, allowing the integration of new NLP tools. However, this does not necessarily mean that the TextFlows platform is the only system that could be used as an open online NLP evaluation framework. In this section, we compare how the features of the TextFlows platform that allow the creation, publishing, and sharing of evaluation workflows compare to other NLP-focused platforms mentioned in Sect. 2. More specifically, we compare TextFlows to other platforms across the following dimensions: Supported languages, Licensing, GUI, Workflow construction and sharing, support for deep learning and POS tagger availability. The comparison is presented in Table 1.

All the platforms support English and several of them support multiple languages, although in most platforms the number of non-English components is much smaller than the number of components for English NLP. Availability of the code and a permissible license allowing the usage and modification of the code are in our opinion essential for easy and transparent integration of new NLP tools that need to be evaluated. When it comes to code availability, all but three platforms (WebLicht, CLAMP, and ELG) are open source. Nevertheless, when it comes to the license under which the code can be used, one can see that TextFlows has the most permissible one, i.e. the MIT license, which allows usage, copying, modification, merging, publishing, and distribution of the code without limitations. Most other platforms, however, also allow the usage of their code under a free and open-source software licensing agreement, such as the Apache license, which also allows integration of new NLP tools.

Although all platforms in Table 1 support some kind of workflow construction (i.e. combining components into processing pipelines), the sharing of created workflows, which is in our opinion essential for the reproducibility of evaluation experiments and results, is less common. Only seven out of eleven compared platforms[12]) offer workflow sharing. This is somewhat surprising, as workflow sharing is an important reason for building such systems. As far as we are aware, only TextFlows, Langage Grid, and KNIME allow to share workflows together with the uploaded data.

Most platforms also do not contain neural components, most likely due to the high computational demand of the underlying models of these components. The only three platforms that contain neural components are ScispaCy, ELG and KNIME. We also compare the platforms according to the availability of POS taggers, which is a use case on which we focus in this study. As one can see from Table 1, all but one platform (LAPPS Grid) support POS tagging (and also other most common NLP operations, such as NER and tokenization), which is not surprising, due to POS tagging being one of the most common NLP processing steps. There is however a distinction between different platforms when it comes to specialized components for text

acquisition, organization and processing. Platforms focused strictly on NLP (e.g., TextFlows, GATE, Language Grid) tend to contain more specialized widgets for corpus acquisition and organization (e.g., widgets that support the acquisition of corpora in different formats) than more general data mining platforms such as GATE.

4 Motivation for the use cases

As stated in Sect. 1, the main motivation for this study is to explore the possibility of using the TextFlows platform as an online NLP evaluation framework. The NLP literature distinguishes between intrinsic and extrinsic evaluation of algorithms (Molla & Hutchinson, 2003). In the intrinsic evaluation, the performance of an algorithm (e.g. a POS tagger) is evaluated with respect to a gold standard, which is often a human-annotated or at least human-verified annotated corpus. On the other hand, in the extrinsic evaluation (which is sometimes also called the "evaluation in use") the performance of an algorithm is measured indirectly, in the context of a larger task, and tells us how the algorithm—acting as a component of a larger workflow—affects the overall workflow performance. We explore the suitability of TextFlows for integration of workflows for both evaluation scenarios.

As a use case, we opted for the evaluation of POS taggers, since POS tagging is one of the most commonly employed preprocessing operations in NLP and since several commonly used open-source POS taggers are integrated in the TextFlows platform. Most of the POS tagger algorithms evaluated in this paper have been subjected to some form of intrinsic evaluation in the past. We were able to find studies that tested the Stanford tagger on different data sets, which could be due to the popularity of this tool. Pinto et al. (2016) compared seven tools for POS tagging (including the Stanford POS tagger and the Maximum entropy POS tagger (Honnibal & Duong, 2012) which are also evaluated in this paper) on two different test sets, one being a corpus of newspaper articles and the other a social network text. Only the pretrained POS taggers were used in the evaluation.

In the study by Yumusak et al. (2014), three NLTK POS taggers (that is, the default tagger, the regex tagger, and the n-gram tagger) were compared for accuracy in three different corpora, that is, Brown (Nelson, 1964), Penn Treebank (Marcus, Santorini, and Marcinkiewicz, 1993) and NPS Chat corpora (Forsyth, Lin, and Martell, 2008). However, POS tagger accuracy was only measured in the same corpus in which they were trained, splitting the corpus into training set (80%) and test set (20%), without using cross-validation. In a study by Horsmann et al. (2015), the accuracy of several POS taggers was measured on different gold standard corpora.
The study covers only one of the POS taggers (the Stanford POS tagger) evaluated in this paper, while the documentation on the accuracy of other open-source POS taggers on various corpora is lacking.

Various researchers have problematized the testing of accuracy of algorithms on just one data set with respect to the generality of conclusions (Demšar, 2006), hence the results of such tests should be taken with reservations. Some studies suggest that the reported accuracies of state-of-the-art POS taggers (which are

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

usually around 97%-98%) would turn out unrealistic in real-life scenarios since the reported accuracies were measured in artificial cross-validation conditions (Giesbrecht & Evert, 2009). Different accuracies were reported for the same POS tagging tools, when tested on different text genres, suggesting that POS taggers are genre-dependent. This becomes particularly clear if we use these POS taggers on informal texts on social networks, where they achieve at most 93% precision (Giesbrecht & Evert, 2009).

We investigate if some of the deficiencies of previous evaluation scenarios can be overcome by using the TextFlows platform. To this end, the aim is to develop evaluation workflows for comparative intrinsic evaluation of seven[13] taggers on six datasets. Another important option we investigate is tagger training inside the TextFlows platform. This allows for a comparison of different algorithms in a scenario where the performance is not influenced by differences in the training data. Finally, we also investigate the platform's capability to handle ensemble scenarios in which different taggers are sequentially connected, and the evaluation of taggers in the extrinsic scenario.

By conducting and publishing these experiments, we wish to confirm or deny a hypothesis that the TextFlows platform allows for a thorough evaluation of different NLP tools in several evaluation scenarios. By publishing these evaluation workflows, we also wish to showcase that all these evaluation experiments in the TexFlows platform are reproducible and publicly available.

5 POS tagger evaluation use case

This section describes the available POS taggers, followed by the integration and evaluation of the POS tagger in the English intrinsic and extrinsic evaluation settings.

5.1 Evaluated POS taggers

The TextFlows platform contains several different POS taggers from the NLTK library (Bird, 2006), which we evaluated in terms of their accuracy. Besides the evaluation of existing four POS taggers in TextFlows, we integrated three new English POS taggers: the Maximum entropy POS tagger that was the default tagger in the NLTK 2.7 library, the Averaged perceptron POS tagger that is the default choice in NLTK 3.2.1 used in the TextFlows platform, as well as the Stanford POS tagger, which is written in Java and accessible in NLTK through a Python wrapper class.

The below list introduces the entire list of POS taggers currently integrated in TextFlows and used in the experiments:

- **ClassBasedTagger:** the Classifier-based POS tagger (Bird, 2006), which is in this paper used with the SVM classifier from the Scikit-learn library (Pedregosa et al., 2011b),

- **NgramTagger:** the N-gram based POS tagger (Bird, 2006),
- **AffixTagger:** the Affix POS tagger (Bird, 2006),
- **BrillTagger:** the Brill POS tagger (Brill, 1992),
- **MaxEntTagger:** the Maximum entropy POS tagger (Honnibal & Duong, 2012),
- **PerceptronTagger:** the Averaged perceptron POS tagger (Honnibal & Duong, 2013), and
- **StanfordTagger:** the Stanford POS tagger (version 3.5.2) (Toutanova, Klein, Manning, and Singer, 2003).

The version of StanfordTagger in the NLTK library is not trainable since it comes with an inbuilt model, so its algorithm is not directly comparable to other algorithms. Therefore, it was not used in some experiments. There is one more POS tagger integrated in the TextFlows platform, a Default POS tagger (**DefaultTagger**), which tags all the words with the same predefined tag (usually the noun tag). Some of the POS taggers in TextFlows allow chaining, which means that they are put in a chain of backoff taggers, which try to tag words that the previous taggers in the chain were unable to tag. The above-mentioned DefaultTagger is normally used in the last position of the chain, as a default choice for the words left untagged.

5.2 Intrinsic POS tagger evaluation

We wanted to test the POS taggers in several settings to showcase a variety of different evaluation possibilities the TextFlows platform offers and to obtain reliable results for comparison. For this purpose, we gathered six different gold standard corpora, all manually annotated (or human-verified) with the Penn Treebank tags (Marcus, Santorini, and Marcinkiewicz, 1993) (see Table 2 for details about the corpora).

We also used the sample of Penn Treebank Wall Street Journal available in NLTK (**PennWSJ-NLTK**) that contains 3,914 sentences (100,676 tokens) and was mostly used for training the POS taggers. PennWSJ-NLTK was chosen as the default training corpus because different PennWSJ corpus samples or the corpus as a whole have been used as such often in the literature (Manning, 2011; Toutanova, Klein, Manning, and Singer, 2003).

To showcase the flexibility of the TextFlows platform, when it comes to support of different evaluation scenarios, we first identify factors expected to influence the accuracy of the POS taggers (i.e. the size of the training set, the text type of

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

Table 2 Goldstandardcorpora,allmanuallyannotated(orhuman-verified)withthePennTreebanktags

Corpus	Num.tokens	Content
GENIA(Kim,Ohta,Tateisi,andTsujii, 2003)	487,097	Abstractsofscientificpapersfromthefieldofmolecularbiology
GUM(Zeldes, 2017)	44,079	Conversational(interviews),narrative(news),informative(travelguides)andinstructional(how-tos)
NAIST(Neubigetal., 2014)	23,158	TEDtalks
BNC-1000(Foster&vanGenabith, 2008)	28,311	,000manuallytaggedsentencesfromtheBritishNationalCorpus(Foster&vanGenabith, 2008)
QuestionBank(Judge,Cahill,andvanGenabith, 2006)	37,831	,000questionsfromtheQuestionBankcorpus(Judge,Cahill,andvanGenabith, 2006)
Chat(Forsyth,Lin,andMartell, 2008)	45,010	,000postsfromage-specificchatrooms

10

1 4

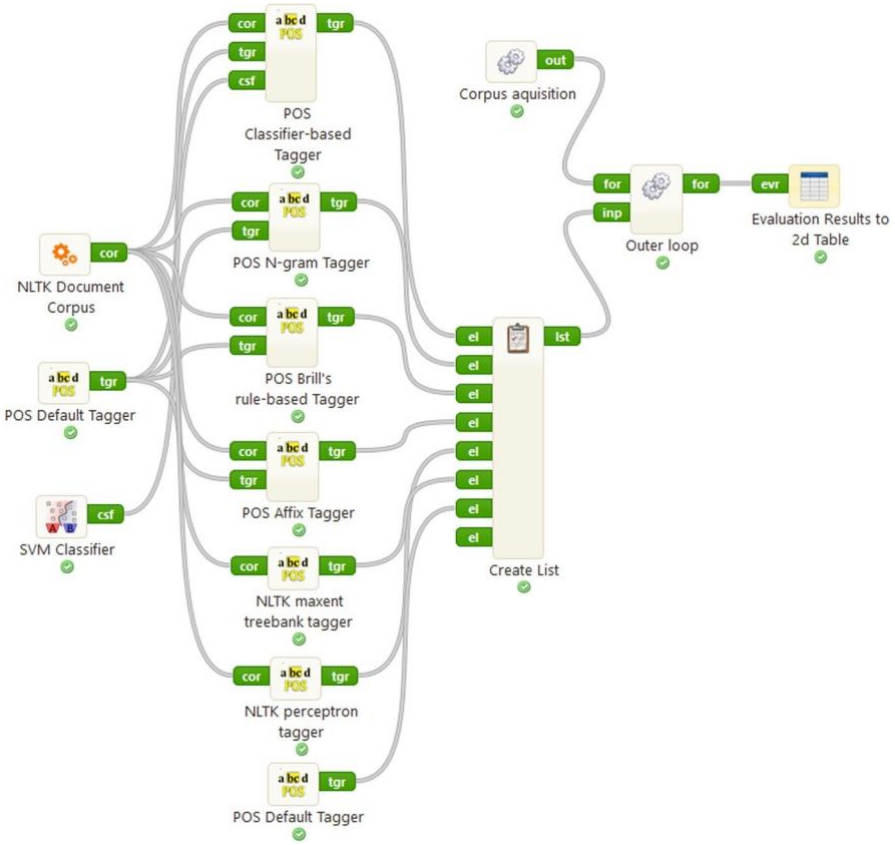

Fig. 4 The workflow implemented in the TextFlows platform, used for the evaluation of six different POS taggers (and the DefaultTagger, which is used as a baseline). We first train all the POS taggers on the PennWSJ-NLTK corpus (*NLTK Document Corpus* widget) and add them to a list of POS taggers. In the *Corpus acquisition* subprocess, we create a list of gold standard corpora. The two lists (POS taggers and corpora) are passed as arguments to the *Outer loop* widget (which contains the nested *Inner loop* shown in Fig. 5). In the *Outer loop* we iterate through the list of corpora and in the *Inner loop* we iterate through POS taggers, in a way that every POS tagger is evaluated on every corpus. The abbreviations on the input and output stubs are as follows: *tgr* (tagger), *el* (element), *lst* (list), *evr* (evaluation results), *out* (output)

the training set, and the algorithm used for training the POS tagger) and then build workflows to evaluate the influence of these factors on the POS tagger accuracy in a set of intrinsic evaluation experiments.

5.2.1 Comparison of POS taggers trained on PennWSJ-NLTK

(Bird, 2006) In our first experiment, we trained all the POS taggers (except StanfordTagger, which is not trainable) on the PennWSJ-NLTK corpus to compare their accuracy on all the other evaluation corpora.

Content courtesy of Springer Nature, terms of use apply. Rights reserved.

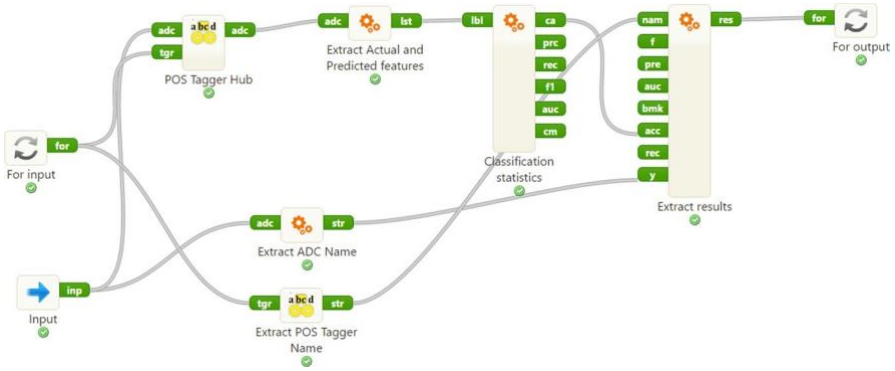

Fig. 5 The inner loop of the workflow for POS tagger accuracy comparison. In the *POS Tagger Hub* widget the corpora are POS-tagged with our taggers which is followed by the extraction of predicted and gold standard POS tags in *Extract Actual and Predicted Features* widget. The accuracy of POS taggers is calculated in the *Classification statistics* widget which is followed by the extraction of the accuracy together with the corpus and the POS tagger name in the *Extract results* widget. The abbreviations on the input and output stubs are as follows: *tgr* (tagger), *adc* (annotated document corpus), *str* (string), *lst* (list), *lbl* (label), *ca/acc* (classification accuracy), *for* (for loop), *inp* (input), *res* (result), *nam* (column name), *y* (row name)

Table 3 POS tagger accuracy evaluation on 6 different test sets (training corpus: PennWSJ-NLTK)

POS tagger	BNC-1000	GENIA	GUM	NAIST	Question Bank	Chat
ClassBasedTagger	84.91%	73.05%	85.02%	89.64%	74.82%	54.43%
NgramTagger	72.54%	59.51%	74.67%	82.74%	67.78%	46.39%
BrillTagger	57.47%	57.22%	54.10%	53.03%	44.03%	26.93%
AffixTagger	24.37%	29.72%	22.91%	19.42%	17.86%	8.53%
PerceptronTagger	**88.20%**	**80.60%**	**88.48%**	**92.21%**	**78.88%**	**58.36%**
MaxEntTagger	86.50%	78.46%	85.48%	89.91%	72.56%	47.91%
DefaultTagger	14.28%	29.14%	12.91%	11.80%	15.66%	8.87%

Best result per dataset is shown in bold

For our experiments, we developed three new workflow components (widgets) in TextFlows: a component used for loading a file in the Penn Treebank format, a component that converts this format into the ADC format used in TextFlows, and a component that compares correctly assigned tags from the gold standard corpora with the tags predicted with our POS taggers, so that we could later measure the taggers' accuracy. All these new widgets are available online in the public TextFlows instance. We included these components into a newly created POS tagger comparison workflow shown in Fig. 4, which is also publicly available online.[15] Fig. 5 shows the inner loop of the workflow.

4 Influence of the length of the training set on the POS tagger accuracy (training corpus: PennWSJ-NLTK, test corpus: GUM)

POS tagger	1000 sentences	2000 sentences	3000 sentences	3914 sentences
NgramTagger	67.69%	72.04%	73.74%	74.67%
AffixTagger	21.50%	22.49%	22.85%	22.91%
ClassBasedTagger	82.33%	84.11%	84.69%	85.02%
BrillTagger	53.14%	54.06%	55.70%	54.10%
MaxEntTagger	81.07%	84.68%	86.16%	85.48%
PerceptronTagger	**84.06%**	**86.63%**	**88.02%**	**88.18%**
DefaultTagger	12.91%	12.91%	12.91%	12.91%

Best result per each training corpus is shown in bold

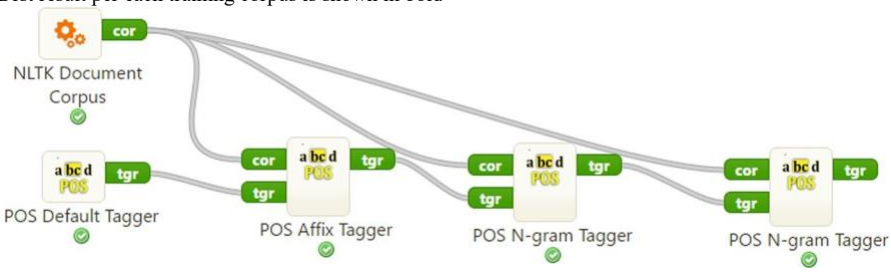

Fig. 6 Implementation of ChainedTagger in TextFlows by putting NgramTagger (n=2), NgramTagger (n=1), AffixTagger and DefaultTagger in a chain of backoff taggers. All taggers in the chain are trained on PennWSJ-NLTK

The results of the experiments are shown in Table 3. We can see that PerceptronTagger has achieved the best results on all the evaluation sets. MaxEntTagger and a ClassBasedTagger with the Support Vector Machine (SVM) classifier achieved slightly worse results. AffixTagger was by far the worst on all the evaluation sets. NgramTagger (with the default n=1, meaning taking two consecutive words) and BrillTagger, though achieving much better results than AffixTagger, also have low accuracies. We can see that all the taggers except AffixTagger achieved better results than DefaultTagger (which tagged all the words with the noun tag) on all the corpora. AffixTagger achieved worse accuracy than DefaultTagger on the Chat corpus.

5.2.2 Effect of training set size on POS tagger accuracy

How the size of the training corpus affects the accuracy of the POS taggers, can be easily tested in TextFlows. For this experiment, we used the PennWSJ-NLTK as a training set. We trained all POS taggers on the first 1,000 sentences of the PennWSJ-NLTK, the first 2,000 sentences, the first 3,000 sentences, and on the whole

PennWSJ-NLTK, which contains 3,914 sentences. We tested these POS taggers on the GUM corpus, which was chosen because it is the most diverse corpus (in terms of different types of texts it contains) out of all our evaluation corpora and therefore

5 Accuracy comparison between the PerceptronTagger and the newly built ChainedTagger (training corpus: PennWSJ-NLTK)

POS tagger	BNC-1000	GENIA	GUM	NAIST	Question Bank	Chat
PerceptronTagger	**90.60%**	82.74%	**91.01%**	**95.05%**	**81.44%**	**61.17%**
ChainedTagger	84.20%	**84.84%**	84.62%	89.65%	75.48%	52.80%

Best result per dataset is shown in bold

Table 6 Accuracy comparison between the pretrained POS taggers (marked with "p") and PennWSJNLTK trained POS taggers

POS tagger	BNC-1000	GENIA	GUM	NAIST	Question Bank	Chat
StanfordTagger(p)	**92.87%**	**97.81%**	**94.00%**	**96.39%**	**89.35%**	58.50%
MaxEntTagger(p)	88.14%	80.64%	88.88%	92.44%	78.77%	**63.16%**
MaxEntTagger	86.50%	78.46%	85.48%	89.91%	72.56%	47.91%
PerceptronTagger	88.05%	80.35%	88.38%	92.08%	78.72%	56.68%
PerceptronTagger(p)	90.60%	82.74%	91.01%	95.05%	81.44%	61.17%

Best result per dataset is shown in bold

most appropriate for the generalization of results. The workflow for this experiment is publicly available online.[14]

Results in Table 4 show that increasing the size of the training corpus in general positively affects the accuracy of the POS taggers. We can also conclude that the accuracy improvement is not linear with the enlargement of the corpora and that it gradually decreases with the size of the training corpus.

5.2.3 POS tagger chain construction and evaluation

The TextFlows platform also supports some ensemble scenarios. Some POS taggers in the NLTK library have a possibility of a backoff (a POS tagger that tags tokens that the main tagger was unable to tag), which allows for the creation of a tagger chain. This functionality is also available for these taggers in TextFlows. NgramTagger and AffixTagger both have the possibility of a backoff, which PerceptronTagger and MaxEntTagger do not, so we decided to put them in a tagger chain (Fig. 6). In our experiments with different parameter settings, we noticed that by building a chain of NgramTagger with n=2, NgramTagger with n=1, AffixTagger, and a DefaultTagger, we get a tagger that is not only fast but also achieves results comparable to our best three POS taggers. Though this new Chained POS tagger (**ChainedTagger**) did not achieve better results than our best-ranked tagger (PerceptronTagger) on most corpora, it did achieve the best results on the GENIA

corpus with 84.84% classification accuracy when trained on the whole PennWSJ-NLTK. The results are shown in Table 5 and the workflow is publicly available online.[17]

7 POS tagger evaluation on the lemmatization task tested on the GUM corpus

POS tagger	Recall	Precision	F-score	Accuracy
StanfordTagger(p)	**0.90**	**0.97**	**0.93**	**90.47%**
ClassBasedTagger	**0.90**	0.96	0.92	90.08%
ChainedTagger	**0.90**	0.96	0.92	90.09%
MaxEntTagger(p)	**0.90**	**0.97**	0.92	89.86%
MaxEntTagger	0.89	**0.97**	0.92	89.38%
PerceptronTagger(p)	**0.90**	**0.97**	**0.93**	90.28%
PerceptronTagger	**0.90**	**0.97**	0.92	89.85%
DefaultTagger	0.83	0.95	0.87	83.16%

Best result per evaluation measure is shown in bold

5.2.4 Evaluation of the pretrained POS taggers

Training of the POS taggers can be time-consuming and it also requires a sufficiently large corpus (as can be seen from our previous experiments). Therefore sometimes a pretrained POS tagger—which was trained in advance on a big corpus—might achieve better accuracy. The TextFlows platform currently offers three POS taggers with a pretrained model that we tested on our corpora. In NLTK 2.7 there was a default MaxEntTagger with a pretrained module that was trained on the PennWSJ corpus which could be called by an "nltk.pos_tag" function. In NLTK 3.0 this POS tagger was replaced by PerceptronTagger. We also tested pretrained StanfordTagger written in Java and accessible through a Python wrapper class. We tested these three taggers on the six evaluation sets and compared their accuracies with our best trained POS taggers. The results are shown in Table 6 and the workflow for this experiment is publicly available online.[15]

5.3 Extrinsic POS tagger evaluation

We have seen above that the TextFlows platform can be used in several intrinsic evaluation scenarios. To demonstrate that the platform also supports extrinsic evaluation, we decided to evaluate the taggers' influence on the lemmatization task, by measuring the effect of different part-of-speech taggers on the lemmatization accuracy.

Lemmatization is a task of attributing a base word form (word type) to each word token (e.g., for a noun in a plural form a word type in singular form is returned). One of the commonly used lemmatizers is the WordNet lemmatizer from the NLTK library (Miller, 1995) that uses a POS tag of a word as an additional input

argument for word lemmatization. In this experiment, we were interested how and if the different POS tagging algorithms influence the accuracy of the WordNet lemmatizer, showing the platform's capability to support extrinsic evaluation scenarios. We tested five taggers that proved most accurate in the previous experiments (Sect. 5.2): StanfordTagger, ClassBasedTagger (used with SVM classifier from Scikit-learn library), ChainedTagger, MaxEntTagger and PerceptronTagger. We also used DefaultTagger (which tagged every word with a noun tag) as a baseline. As mentioned before, the NLTK version of StanfordTagger that we employ can not be trained and ClassBasedTagger and ChainedTagger do not have a pretrained model, so these taggers were only used in the experiment once, while PerceptronTagger and MaxEntTagger were used in the experiment twice, once using a pretrained model and once being trained on the PennWSJ-NLTK. PennWSJ-NLTK was also used for training of ClassBasedTagger and ChainedTagger. For lemmatizer accuracy evaluation we used the GUM corpus, which contains lemmas manually verified by a human annotator that were used as a gold standard in this experiment. The results of this experiment are presented in Table 7 and the workflow is publicly available online.[19]

6 Conclusions

This paper demonstrates how the open-source web platform TextFlows can be exploited as an environment for NLP tools evaluation. Existing workflows and components can be reused and upgraded, providing an environment for transparent and replicable experiments, which are at the core of the open science approach.

We evaluated and compared seven open-source tools for part-of-speech tagging on six different corpora, analyzing the effects of different factors that may affect the accuracy of the part-of-speech tagging tools, such as, for example, the training corpus length. We also evaluated the POS taggers extrinsically, on the lemmatization task, where the POS tag is used as an additional input. By doing this, we showcased that the TextFlows platform supports a variety of different evaluation scenarios. The ease of integration of new NLP tools, the reproducibility of experiments, and the reusability offered by the workflow sharing and publishing functionality, demonstrate that the platform can offer a viable framework for online NLP tools evaluation. Nevertheless, as one can see from Table 1, where we compared eleven platforms across several dimensions that affect the scope and ease of use and reproducibility, several other platforms besides TextFlows would also be appropriate to be used as an online evaluation framework. Among them, especially KNIME seems to be an adequate choice, since it offers all the functionalities of the TextFlows platform and also contains some novel neural state-of-the-art models for text processing.

While the TextFlows platform in our opinion offers numerous state-of-the-art features, i.e. a web-browser use that requires no installation, the ease of sharing and publicizing workflows, and the visual programming functionality, the platform

6083/

nevertheless has some limitations. In this research, we have focused on the integration and evaluation of open-source taggers from popular NLP libraries, such as NLTK, mostly for reasons of convenience, since these tools have been developed with a specific focus on the ease of integration in larger NLP systems or workflows and were therefore easy to adapt for our specific use case, but also because of the limitations of the TextFlows platform mentioned below.

6.1 Limitations of the platform

The TextFlows platform has several limitations when it comes to the integration and evaluation of state-of-the-art neural NLP tools, which tend to be trained on very large training corpora and use substantial amounts of computational resources. The platform in its current form lacks the computational capacity for training large neural models due to RAM limitations and does not support accelerated parallel computation on Graphics Processing Units (GPUs). While using pre-trained neural models in the inference setting (e.g., for POS tagging) is less problematic from the standpoint of available compute power, this functionality is currently still not implemented due to the compatibility issues with the underlying ADC tokenization format, which only allows for word-level tokenization. With current trends in embeddings representations and transformer architectures, subword-level tokenization is becoming predominant, and therefore token-level representations are becoming insufficient. Most of these problems stem from the fact that the development of TextFlows (and its parent ClowdFlows platform) started before neural methods became predominant in NLP.

Another limitation of the platform is its current lack of support for languages other than English. Most of the components (i.e. widgets) were built specifically for the usage in English NLP. While there are several general components (e.g., widgets for classification, evaluation, post-processing, etc.) that can be used for any language, the preprocessing tools for POS tagging, lemmatization, stemming, stopword removal, and tokenization only support English. This means that the usage of the platform for languages other than English is severely limited.

6.2 Future work

To solve these limitations, i.e. the lack of support for bigger corpora, computationally expensive models, and subword representations, we are currently working on the merging of widgets from TextFlows and the existing data mining platform ClowdFlows Kranjc et al. (2012) into a new, more robust and scalable version of ClowdFlows (Žnidaršic, Podpečan, Kranjc, and Pollak, 2024, submitted) built on a new architecture with a separated front-end and back-end, to support easier widget implementation, faster workflow execution, and support integration of neural architectures by the introduction of new representation formats based on subwords,

together with the system's installation of the public online instance of the platform on more capable servers with more computational resources.

The future extension of the platform will therefore focus on the expansion of the available NLP preprocessing (and classification) tools with state-of-the-art neural options and other supporting widgets, mostly for corpus analysis and evaluation, and on the expansion of widgets for languages other than English. We also plan to upgrade current evaluation widgets to support other evaluation criteria, such as AUC, F-score and run time in all our tests. In this work, we used the Penn Treebank part of speech tag set only, since all of the tagged corpora that we used as gold standards were annotated with these tags. In the future, we plan to extend the evaluation to the corpora annotated with different tag sets, such as universal dependencies.

Finally, integration of the components from recent European initiatives, such as the dockerized tools being made available in the scope of the European language grid (Rehm, Berger, et al., 2020) will be considered.

Acknowledgements The authors acknowledge the financial support obtained from the Slovenian Research Agency for Research core funding for the program Knowledge Technologies (No. P2-0103).

Declaration

Conflict of interest The authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Bird, Steven, Edward Loper, and Ewan Klein. 2009. Natural Language Processing with Python. O'Reilly Media.
2. Bontcheva, Kalina, Hamish Cunningham, and Valentin Tablan. "Web-based collaborative annotation for NLP." Proceedings of the Seventh International Conference on Language Resources and Evaluation (LREC'10). 2010.
3. Brill, Eric. "A simple rule-based part of speech tagger." Proceedings of the workshop on Speech and Natural Language. Association for Computational Linguistics, 1992.
4. Carpenter, Bob. "Lingpipe for 99.99% recall of gene mentions." Proceedings of the Second BioCreative Challenge Evaluation Workshop. 2007.
5. Cunningham, Hamish. "GATE, a general architecture for text engineering." Computers and the Humanities 36, no. 2 (2002): 223-254.
6. Demšar, Janez. "Statistical comparisons of classifiers over multiple data sets." Journal of Machine Learning Research 7 (2006): 1-30.

7. Feldman, Ronen, and James Sanger. The Text Mining Handbook. Cambridge University Press, 2007.
8. Foster, Jennifer, and Josef van Genabith. "Parser evaluation and the BNC: Evaluating 4 constituency parsers with 3 metrics." Proceedings of LREC'08. 2008.
9. Giesbrecht, Evelyn, and Stefan Evert. "Part-of-speech (POS) tagging – a solved task? An evaluation of POS taggers for the web as corpus." Proceedings of the Fifth Web as Corpus Workshop (WAC5). Association for Computational Linguistics, 2009.
10. Honnibal, Matthew, and Le Viet Duc. "Maximum entropy POS tagger." GitHub repository. 2012.
11. Darshan Bhavesh Mehta. (2024). NLP Enhanced IVR Routing for Electric Vehicle Fleet Operations Utilizing Solar Charging Infrastructure. International Journal of Communication Networks and Information Security (IJCNIS), 16(5), 978–991
12. Honnibal, Matthew, and Le Viet Duc. "Averaged perceptron tagger." GitHub repository. 2013.
13. Horsmann, Tobias, Nicolas Erbs, and Torsten Zesch. "Fast or accurate? A comparative evaluation of POS tagging models." Proceedings of GSCL. 2015.
14. Ide, Nancy, and James Pustejovsky, eds. Handbook of Linguistic Annotation. Springer, 2007.
15. Ishida, Toru. "Language grid: An infrastructure for intercultural collaboration." Proceedings of the 2006 Symposium on Applications and the Internet. IEEE, 2006.
16. Kim, Jin-Dong, Tomoko Ohta, Yuka Tateisi, and Jun'ichi Tsujii. "GENIA corpus–a semantically annotated corpus for bio-textmining." Bioinformatics 19, no. suppl_1 (2003): i180-i182.
17. Kingsbury, Paul, and Martha Palmer. "PropBank: The next level of treebank." Proceedings of Treebanks and Linguistic Theories. 2002.
18. Kranjc, Janez, Vid Podpečan, and Nada Lavrač. "ClowdFlows: A cloud based scientific workflow platform." ECML PKDD. Springer, 2012.
19. Lafferty, John, Andrew McCallum, and Fernando Pereira. "Conditional random fields: Probabilistic models for segmenting and labeling sequence data." Proceedings of the International Conference on Machine Learning (ICML). 2001.
20. Marcus, Mitchell P., Beatrice Santorini, and Mary Ann Marcinkiewicz. "Building a large annotated corpus of English: The Penn Treebank." Computational Linguistics 19, no. 2 (1993): 313-330.
21. Miller, George A. "WordNet: A lexical database for English." Communications of the ACM 38, no. 11 (1995): 39-41.
22. Molla, Diego, and Ben Hutchinson. "Intrinsic versus extrinsic evaluations of parsing systems." Proceedings of EACL. 2003.
23. Pang, Bo, Lillian Lee, and Shivakumar Vaithyanathan. "Thumbs up? Sentiment classification using machine learning techniques." Proceedings of EMNLP. 2002.
24. Papineni, Kishore, Salim Roukos, Todd Ward, and Wei-Jing Zhu. "BLEU: a method for automatic evaluation of machine translation." Proceedings of ACL. 2002.
25. Pedregosa, Fabian, et al. "Scikit-learn: Machine learning in Python." The Journal of Machine Learning Research 12 (2011): 2825-2830.
26. Savova, Guergana K., et al. "Mayo clinical Text Analysis and Knowledge Extraction System (cTAKES): architecture, component evaluation and applications." Journal of the American Medical Informatics Association 17, no. 5 (2010): 507–513.
27. Tjong Kim Sang, Erik F. "Introduction to the CoNLL-2000 shared task: Chunking." Proceedings of CoNLL. 2000.
28. Tjong Kim Sang, Erik F, and Fien De Meulder. "Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition." Proceedings of CoNLL. 2003.
29. Toutanova, Kristina, Dan Klein, Christopher Manning, and Yoram Singer. "Feature-rich part-of-speech tagging with a cyclic dependency network." Proceedings of NAACL. 2003.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.